Часть 1. Основы Python

1.1. Переменные и типы данных

К концу занятия ты сможешь:

- хранить данные в переменных и давать им понятные имена;
- определять тип значения: число, текст, «да/нет» или «пусто»;
- менять значения и обменивать две переменные значениями.

Разминка

Ответь устно — это вопросы по темам 0.1–0.3.

1. Python выполняет программу целиком сразу или строка за строкой? Что будет со строками после ошибки?

2. Какой командой запустить файл main.py из терминала?

3. Зачем делать коммит и чем хорошее сообщение коммита отличается от плохого?

Игрок сменил ник. Что делать с кодом?

main.py

```
print("Привет,", "Shadow_Fox")
print("Shadow_Fox", "- добро пожаловать в лигу")
print("Сегодня", "Shadow_Fox", "играет первый матч")
print("Удачи,", "Shadow_Fox")
```

Вывод в консоли

```
Привет, Shadow_Fox
Shadow_Fox - добро пожаловать в лигу
Сегодня Shadow_Fox играет первый матч
Удачи, Shadow_Fox
```

Теперь игрок называет себя **Night_Owl**.

Сколько мест в коде придётся исправить? А если таких строк 50 и одну ты пропустишь?

Сегодня научимся менять такое в одном месте.

План занятия

Урок	Что изучаем	Практика
Урок 1	Переменные: что это, как называть, константы	«Сколько я прожил»
Урок 2	Типы данных: int, float, str, bool, None, type()	«Карточка персонажа»
Урок 3	Несколько переменных сразу, обмен значений	«Смена сторон», мини-проект «Поход на концерт»

После каждого блока — вопросы и практика. Задания со ★ — для тех, кто успел раньше.

Вспомним: print и комментарии (0.2)

main.py

```
# Это комментарий: Python его пропускает
print("Уровень", 7)
print("Монеты:", 350, "шт.")
```

Вывод в консоли

```
Уровень 7
Монеты: 350 шт.
```

Через запятую в print можно передать несколько значений.

Между ними Python сам ставит один пробел.

Текст пишем в кавычках, числа — без кавычек.

Строка после # — комментарий для человека, Python её не выполняет.

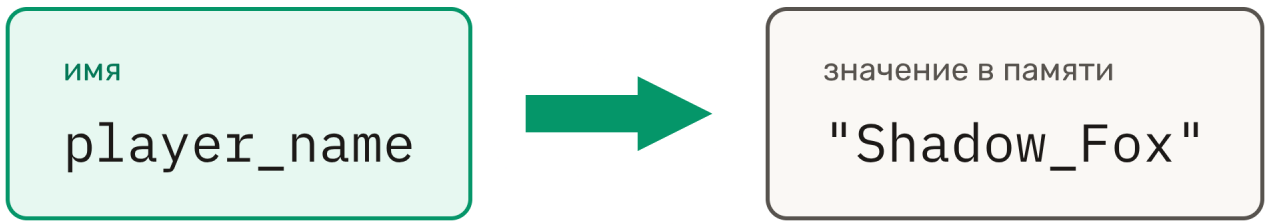
Урок 1

Блок 1 из 3. Переменные и имена

В этом блоке ты научишься сохранять значение под именем, менять его и называть переменные так, чтобы код было легко читать.

Что такое переменная

Переменная — это имя, подписанное на значение. Python хранит значение в памяти компьютера, а по имени находит его, когда оно понадобится.



Похоже на ник в игре: сервер хранит твои очки и предметы, а по нику их находит. Сменить ник — значит переклеить подпись, данные остаются те же.

Зачем: ник меняется в одном месте

Было: ник в каждой строке

```
print("Привет,", "Shadow_Fox")
print("Shadow_Fox", "- добро пожаловать в лигу")
print("Сегодня", "Shadow_Fox", "играет первый матч")
print("Удачи,", "Shadow_Fox")
```

Вывод в консоли

```
Привет, Shadow_Fox
Shadow_Fox - добро пожаловать в лигу
Сегодня Shadow_Fox играет первый матч
Удачи, Shadow_Fox
```

Новый ник — правим 4 строки.

Стало: ник в переменной

```
player_name = "Night_Owl" # меняем только здесь
print("Привет,", player_name)
print(player_name, "- добро пожаловать в лигу")
print("Сегодня", player_name, "играет первый матч")
print("Удачи,", player_name)
```

Вывод в консоли

```
Привет, Night_Owl
Night_Owl - добро пожаловать в лигу
Сегодня Night_Owl играет первый матч
Удачи, Night_Owl
```

Новый ник — правим 1 строку.

Как записывается присваивание



- Это **присваивание**: переменная получает значение. Если её не было — она создаётся.
- Знак = здесь не «равно». Сначала Python вычисляет правую часть, потом подписывает результат именем слева.
- Имя всегда слева, значение справа. Проверку «равно ли» пишут двумя знаками == — это тема 1.4.

Пример: данные игрока

main.py

```
# Данные игрока
player_name = "Shadow_Fox"
level = 7
coins = 350

print("Игрок:", player_name)
print("Уровень:", level)
print("Монеты:", coins)
```

Вывод в консоли

```
Игрок: Shadow_Fox
Уровень: 7
Монеты: 350
```

Строки 2–4 создают три переменные.

Строки 6–8: имя без кавычек — Python подставляет значение.

Сравни: с кавычками это уже просто текст.

Код	Выведет
`print(player_name)`	`Shadow_Fox`
`print("player_name")`	`player_name`

Правило: имя переменной в коде пишется без кавычек.

Новое значение заменяет старое

main.py

```
score = 0
print("Очки:", score)
score = 10
print("Очки:", score)
score = score + 5
print("Очки:", score)
```

Вывод в консоли

```
Очки: 0
Очки: 10
Очки: 15
```

Строка	score	Что произошло
1	0	переменная создана
3	10	старое значение 0 забыто
5	15	сначала справа: 10 + 5, потом результат записан в score

Правило: переменная помнит только последнее присвоенное значение.

score = score + 5 — не «уравнение», а команда: «возьми старые очки, прибавь 5, запиши обратно».

Как называть переменные

Правила Python

- буквы, цифры и знак подчёркивания _
- не начинается с цифры, без пробелов
- регистр важен: score и Score — разные имена

Договорённость программистов

- **snake_case**: маленькие буквы, слова через _ (player_name, total_score)
- английские слова со смыслом: coins, а не a или x1

main.py

```
score = 10
Score = 99
print(score, Score)
```

Вывод в консоли

```
10 99
```

Две разные переменные. Так писать можно, но не нужно: легко перепутать.

Ловушка: имя по неправильным правилам

Имя начинается с цифры

main.py

```
2nd_place = "Ника"
print(2nd_place)
```

Вывод в консоли

```
  File ".../main.py", line 1
    2nd_place = "Ника"
    ^
SyntaxError: invalid decimal literal
```

В имени пробел

main.py

```
player name = "Ника"
```

Вывод в консоли

```
  File ".../main.py", line 1
    player name = "Ника"
           ^^^^
SyntaxError: invalid syntax
```

SyntaxError — ошибка в записи: Python не понял код и не выполнил ни одной строки. Исправление: second_place, player_name.

Ловушка: опечатка в имени

```
main.py
player_name = "Shadow_Fox"
print(player_nmae)
```

Переменной player_nmae нет: в имени переставлены буквы. Программа запустилась и упала на строке 2.

```
Вывод в консоли
Traceback (most recent call last):
  File ".../main.py", line 2, in <module>
    print(player_nmae)
          ^^^^^^^^^^^
NameError: name 'player_nmae' is not defined. Did you mean: 'player_name'?
```

Traceback — сообщение об ошибке. Читай его **снизу вверх**: последняя строка — тип ошибки и что случилось («имя не определено, может, ты имел в виду player_name?»), выше — строка кода и ^ под ошибкой, ещё выше — файл и номер строки.

Константы: имя заглавными

Константа — значение, которое не должно меняться во время работы программы: часов в сутках, максимальный уровень в игре. Её имя пишут ЗАГЛАВНЫМИ буквами.

main.py

```
MAX_LEVEL = 50
HOURS_IN_DAY = 24
print("Максимальный уровень:", MAX_LEVEL)
print("Часов в сутках:", HOURS_IN_DAY)
```

Вывод в консоли

```
Максимальный уровень: 50
Часов в сутках: 24
```

Зачем: в коде 24 — просто число, а HOURS_IN_DAY сразу объясняет, что это.

Граница: Python не запретит изменить MAX_LEVEL. Заглавные — договорённость между программистами: «это не трогай».

Правило: число со смыслом — в константу с понятным именем.

Живой кодинг: объявление о турнире

Пишем вместе по шагам

1. Переменные: название игры, день, время начала, приз в монетах.
2. Константа MAX_PLAYERS — сколько мест на турнире.
3. Выводим объявление через print.
4. Ищем ошибку по traceback.
5. Переносим турнир на другой день — меняем одну строку.

Место для скриншота: VS Code — файл main.py и вывод в терминале

Вопросы по блоку 1

1. Что выведет программа и почему?

```
coins = 5
coins = coins * 2
coins = 3
print(coins)
```

2. Найди ошибки: my score = 10 и 1st_player = "Аня"

3. Чем отличается print(level) от print("level")?

4. Python не запрещает менять MAX_LEVEL. Зачем тогда писать имя заглавными?

Задание: «Сколько я прожил»

- Заведи переменную со своим возрастом в годах.
- Заведи константы: дней в году, часов в сутках, минут в часе, секунд в минуте.
- Посчитай, сколько это примерно дней, часов и секунд.
- Выведи 4 строки: возраст в годах, днях, часах и секундах.

Подсказка: * — умножение, как в калькуляторе Python из темы 0.1. Часы = дни * HOURS_IN_DAY. Результат можно сразу положить в новую переменную.

★ **Успел раньше:** добавь строку с возрастом в минутах.

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

Что должно получиться

Вывод в консоли (возраст 15 лет)

```
Мне 15 лет
Это примерно 5475 дней
Или 131400 часов
Или 473040000 секунд
```

Проверь себя

- Поменял возраст в одной строке — пересчитались все строки вывода.
- Числа 365, 24 и 60 встречаются только в константах.
- Все имена в snake_case, константы — заглавными.

★★ «Предскажи, потом проверь»

Не запуская код, запиши в тетради значения всех переменных после каждой строки. Потом запусти и сравни. Где ошибся — найди почему.

main.py

```
coins = 100
bonus = coins
coins = coins + 50
bonus = bonus + coins
gems = bonus - coins
coins = gems
print(coins, bonus, gems)
```

Пример таблицы в тетради

Строка	coins	bonus	gems
1	100	—	—
2	?	?	—
3	?	?	—
…			

Прочерк — переменной ещё нет.

Урок 2

Блок 2 из 3. Типы данных

В этом блоке ты научишься различать целые и дробные числа, текст, «да/нет» и «пусто» и узнавать тип любого значения.

Зачем нужны типы

Тип — вид значения: число, текст и так далее. От типа зависит, что с значением можно делать. Как в жизни: цены в чеке складывают, а цифры номера телефона — нет.

main.py

```
print(5 * 2)
print("5" * 2)
```

Вывод в консоли

```
10
55
```

5 — число: умножили на 2, получили 10.

"5" — текст из одного символа: «умножить на 2» значит повторить его 2 раза.

Одна и та же операция по-разному работает с разными типами.

Числа: int и float

int — целое число

Уровень, монеты, счёт матча, номер этажа. Бывает и отрицательным: −3.

float — дробное число

Скорость, рост, оценка трека 4.8. Дробную часть отделяет **точка**: 4.5, а не 4,5.

main.py

```
level = 12 # int: целое число
speed = 4.5 # float: дробное, через точку
temperature = -3 # int бывает и отрицательным
print(level, speed, temperature)
```

Вывод в консоли

```
12 4.5 -3
```

Правило: считаем штуки — int, меряем с дробной частью — float.

Текст: str

str (строка) — любой текст в кавычках: название песни, ник, сообщение. Кавычки можно брать двойные или одинарные, главное — одинаковые в начале и в конце.

main.py

```
song_title = "Звёздный путь"
artist = 'Лиса'
room = "305"
quote = 'Он сказал: "Погнали"'
print(song_title, "-", artist)
print("Кабинет", room)
print(quote)
```

Вывод в консоли

```
Звёздный путь - Лиса
Кабинет 305
Он сказал: "Погнали"
```

"305" — строка, хотя состоит из цифр: номер кабинета не складывают.

Нужны кавычки внутри текста — снаружи возьми кавычки другого вида.

Правило: всё в кавычках — строка.

«Да» или «нет»: bool

bool (логическое значение) — только два варианта: **True** (да, правда) и **False** (нет, ложь). Жив ли персонаж, есть ли ключ, включён ли звук.

main.py

```
is_alive = True
has_key = False
print("Жив:", is_alive)
print("Есть ключ:", has_key)
```

Вывод в консоли

```
Жив: True
Есть ключ: False
```

Ошибка: маленькая буква

```
is_alive = true
Traceback (most recent call last):
  File ".../main.py", line 1, in <module>
    is_alive = true
               ^^^^
NameError: name 'true' is not defined. Did you mean: 'True'?
```

True и False — с большой буквы и без кавычек.

«Пока ничего»: None

None — особое значение «значения пока нет». Рекорд ещё не поставлен, гильдия не выбрана. Пишется с большой буквы, без кавычек.

main.py

```
best_score = None
print("Рекорд:", best_score)
best_score = 0
print("Рекорд:", best_score)
```

Вывод в консоли

```
Рекорд: None
Рекорд: 0
```

None — ещё не играли.

0 — сыграли и набрали ноль. Это уже результат.

"" (пустая строка) — текст, в котором нет символов. Тоже не None.

Правило: None — когда значения нет совсем.

Как узнать тип: type()

type(значение) — функция, которая сообщает тип. Удобно, когда не уверен, что лежит в переменной.

main.py

```
print(type(42))
print(type(3.14))
print(type("42"))
print(type(True))
print(type(None))
```

Вывод в консоли

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
```

<class 'int'> читай как «тип int». NoneType — тип, у которого есть только одно значение: None.

42 и "42" выглядят похоже, но type() показывает разницу.

Тип хранится у значения

Динамическая типизация: тип переменной определяет значение, которое в ней сейчас. Объявлять тип заранее не нужно, и одно имя может получить значение другого типа.

main.py

```
status = 404
print(status, type(status))
status = "не найдено"
print(status, type(status))
```

Вывод в консоли

```
404 <class 'int'>
не найдено <class 'str'>
```

Строка 1: status — число, тип int.

Строка 3: то же имя получило строку — тип стал str.

Граница: так можно, но без причины не делай. Через 50 строк уже не вспомнишь, число там или текст.

Правило: одна переменная — один смысл и один тип.

Ловушка: "2" + "2"

main.py

```
print("2" + "2")
print(2 + 2)
```

Вывод в консоли

```
22
4
```

Строки + склеивает, числа + складывает.

main.py

```
lives = "5"
print(lives + 1)
```

Вывод в консоли

```
Traceback (most recent call last):
  File ".../main.py", line 2, in <module>
    print(lives + 1)
          ~~~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

Строку и число сложить нельзя: «склеивать можно только строку со строкой».

Правило: число в кавычках — это текст. Числа для расчётов пиши без кавычек. Эта ловушка снова встретится в 1.2.

Ловушка: странности дробных чисел

main.py

```
print(0.1 + 0.2)
print(10 / 2)
print(10 // 2)
print(type(10 / 2), type(10 // 2))
```

Вывод в консоли

```
0.30000000000000004
5.0
5
<class 'float'> <class 'int'>
```

0.1 + 0.2: компьютер хранит дробные числа приближённо, поэтому в конце бывает «хвостик». Это не ошибка Python, так во всех языках.

/ — деление, результат всегда float: 5.0, даже если делится нацело.

// — деление нацело, результат int: 5. Подробно — в теме 1.3.

Правило: после / жди float, даже если поделилось ровно.

Живой кодинг: карточка трека в плеере

Пишем вместе по шагам

1. Название трека — str, длительность в секундах — int.
2. Рейтинг — float, «лайк поставлен» — bool.
3. Альбом неизвестен — None.
4. Выводим каждое значение рядом с его типом через type().
5. Ищем ошибку по traceback.

Место для скриншота: VS Code — код карточки трека и вывод с типами в терминале

Вопросы по блоку 2

1. Какого типа каждое значение: 15, 15.0, "15", True, None?

2. Почему print("7" + "3") выводит 73, а не 10?

3. Что будет, если написать is_alive = false? Как это исправить?

4. Найди ошибку. Какая будет ошибка и почему?

```
gold = "120"
print(gold + 30)
```

5. Рекорд игрока 0 или None — в чём разница по смыслу?

Задание: «Карточка персонажа»

- Заведи переменные героя: имя, уровень, здоровье (с дробной частью), золото, «жив ли».
- Добавь гильдию: герой пока ни в какой гильдии не состоит.
- Выведи карточку: каждая характеристика с подписью на своей строке.
- Последней строкой выведи через type() типы имени, здоровья и «жив ли».

Подсказка: «гильдии пока нет» — значение None. True и None пишутся с большой буквы и без кавычек.

★ **Успел раньше:** выведи типы всех шести переменных, а не трёх.

Упала программа — читай traceback снизу вверх и ищи ошибку по сообщению.

Что должно получиться

Вывод в консоли

```
Карточка героя
Имя: Аркадия
Уровень: 5
Здоровье: 87.5
Золото: 120
Жив: True
Гильдия: None
Типы: <class 'str'> <class 'float'> <class 'bool'>
```

Проверь себя

- Тип здоровья — float, а не int или str.
- «Жив» — bool: True без кавычек.
- У гильдии тип NoneType. Если str — ты взял None в кавычки.

★★ «Угадай тип»

Для каждой строки запиши прогноз: какой тип получится и что напечатается. Потом запусти и найди, где ошибся и почему.

main.py

```
print(type(10 // 3))
print(type(7 / 7))
print(type(3 * 2.0))
print(type("7" + "7"))
print(type("ха" * 3), "ха" * 3)
print(type(True + True), True + True)
```

Пример записи в тетради

№	Мой прогноз	Запуск
1	int	?
2	?	?
…		

Самое странное — последняя строка. Придумай, почему так.

Урок 3

Блок 3 из 3. Несколько переменных и обмен

В этом блоке ты научишься создавать несколько переменных одной строкой и менять две переменные значениями без потерь.

Несколько переменных одной строкой

Когда значения связаны по смыслу (координаты x и y, три канала цвета), их удобно создать сразу: короче и видно, что они вместе.

main.py

```
x, y = 100, 250
print("Координаты:", x, y)
red = green = blue = 0
print("Цвет:", red, green, blue)
```

Вывод в консоли

```
Координаты: 100 250
Цвет: 0 0 0
```

x, y = 100, 250 — первое имя получает первое значение, второе — второе. a = b = 0 — одно значение для нескольких имён.

Ошибка: количество не совпадает

```
width, height = 800, 600, 32
Traceback (most recent call last):
  File ".../main.py", line 1, in <module>
    width, height = 800, 600, 32
    ^^^^^^^^^^^^^
ValueError: too many values to unpack (expected 2)
```

«Слишком много значений: ожидалось 2». Слева два имени, справа три значения.

Правило: имён слева столько же, сколько значений справа.

Ловушка: обмен «в лоб» теряет значение

Команды после перерыва меняются сторонами поля. Хотим, чтобы слева стали Совы, а справа — Тигры.

main.py

```
left_team = "Тигры"
right_team = "Совы"
left_team = right_team
right_team = left_team
print(left_team, right_team)
```

Вывод в консоли

```
Совы Совы
```

Строка	left_team	right_team	Что произошло
2	Тигры	Совы	начальные значения
3	Совы	Совы	«Тигры» потеряны
4	Совы	Совы	копируем уже «Совы»

Ошибки Python нет — программа просто делает не то. Это **логическая ошибка**: её видно только по результату.

Обмен значений: два правильных способа

Было: left_team = "Тигры", right_team = "Совы".

Через третью переменную

main.py

```
left_team = "Тигры"
right_team = "Совы"
temp = left_team # запомнили «Тигры»
left_team = right_team
right_team = temp
print(left_team, right_team)
```

Вывод в консоли

```
Совы Тигры
```

Работает в любом языке. Нужна лишняя переменная.

Одной строкой, как в Python

main.py

```
left_team = "Тигры"
right_team = "Совы"
left_team, right_team = right_team, left_team
print(left_team, right_team)
```

Вывод в консоли

```
Совы Тигры
```

Правило: в Python меняй значения так: a, b = b, a.

Почему a, b = b, a ничего не теряет

```
left_team, right_team = right_team, left_team
```

Шаг 1. Правая часть

Python сначала читает оба значения справа и держит их наготове:

`"Совы", "Тигры"`



Шаг 2. Раскладка по именам

Только потом подписывает их именами слева, по порядку:

```
left_team: "Совы"
right_team: "Тигры"
```

В ловушке со слайда 38 каждая строка сразу затирала значение. Здесь затирать нечего: всё прочитано заранее.

Вопросы по блоку 3

1. Что выведет программа?

```
a, b = 3, 8
a, b = b, a
print(a, b)
```

2. Почему обмен двумя строками a = b и b = a не работает?

3. Что будет, если написать width, height = 800, 600, 32?

Задание: «Смена сторон»

- Заведи переменные: команда слева, команда справа, голы левой и голы правой команды.
- Выведи табло первого тайма: «Ракеты 2 : 1 Кометы».
- После перерыва команды меняются сторонами, а голы остаются у своих команд. Обменяй названия и голы.
- Выведи табло второго тайма.

Подсказка: обмен — одной строкой, как на слайде 39. Знак «:» в табло — это просто строка ":" в print.

★ **Успел раньше:** добавь цвет формы каждой команды — он тоже переезжает вместе с командой.

Что должно получиться

Вывод в консоли

```
Первый тайм:
Ракеты 2 : 1 Кометы
Второй тайм:
Кометы 1 : 2 Ракеты
```

Проверь себя

- У Ракет по-прежнему 2 гола, у Комет — 1.
- Ни одно название не потерялось и не повторилось.
- Для обмена не понадобилась лишняя переменная.

★★ «Ротация в волейболе»

Три игрока стоят в трёх зонах площадки. При переходе каждый сдвигается на зону вперёд, а игрок из последней зоны уходит в первую.

Сделай переход **одной строкой**, без дополнительной переменной. Выведи старт и два перехода.

Вывод в консоли

```
Старт: Аня Борис Вика
Переход 1: Вика Аня Борис
Переход 2: Борис Вика Аня
```

Вопросы по всей теме

1. В начале занятия ник Shadow_Fox был записан в коде 4 раза. Как сделать, чтобы он менялся в одной строке? Какого типа будет эта переменная?

2. Код ниже выводит 4. Что будет, если в первой строке поменять 3 на "3"?

```
lives = 3
print(lives + 1)
```

3. В теме 0.2 мы писали print("Привет"). Как print понимает, что в print(score) — имя переменной, а в print("score") — текст?

Мини-проект: «Поход на концерт»

Вы с друзьями идёте на концерт. Программа собирает всё про поход в одну карточку: группа, сколько вас, сколько стоят билеты на всех, куплены ли они.

А в зале вы с другом меняетесь местами — программа это тоже покажет.

Вывод в консоли: от запуска до конца

```
Поход на концерт
Группа: Ночные огни
Нас идёт: 3
Всего за билеты: 4500 руб.
Билеты куплены: True
Мерч: None
Моё место: 12 | место друга: 13
Поменялись местами
Моё место: 13 | место друга: 12
Тип суммы: <class 'int'>
```

Что должна уметь программа

Хранить данные о концерте: группу, цену билета, сколько вас идёт.

Считать, сколько стоят билеты на всю компанию.

Показывать, куплены ли билеты, и что мерч пока не выбран.

Менять местами твоё место и место друга.

Это пригодится в Проекте 1 «Консольные игры»: имя игрока, счёт и число попыток будут храниться в переменных.

План по шагам

Шаг	Что сделать	Тема	Как проверить
1	Переменные: группа, цена билета, сколько вас, мерч (пока нет)	1.1, блоки 1–2	Запусти — ошибок нет, ничего не выводится
2	Сумма = цена * сколько вас; отметка «билеты куплены»	1.1, блоки 1–2	Временно выведи сумму — должно быть 4500
3	Карточка: заголовок и 5 строк с подписями	0.2 и 1.1	Запусти — 6 строк, как в примере
4	Два места одной строкой, вывод, обмен, вывод, тип суммы	1.1, блоки 2–3	Последние строки: «Моё место: 13 | место друга: 12» и тип int

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

Заготовка: замени комментарии кодом

main.py

```
# Шаг 1: данные концерта
# группа (строка), цена билета и сколько вас идёт (целые числа)
# мерч ещё не выбрали — какое значение означает «пока нет»?

# Шаг 2: сумма за билеты на всех — посчитай из переменных
# отметка «билеты куплены» — True или False?

# Шаг 3: карточка — заголовок и строки с подписями через print

# Шаг 4: твоё место и место друга — две переменные одной строкой
# выведи места, поменяй их одной строкой, выведи ещё раз
# последней строкой выведи тип суммы через type()
```

Ты должен суметь объяснить каждую строку своего кода.

Готово, если…

- **Блок 1:** поменял цену билета на 2000 — сумма стала 6000 сама, без правки других строк.
- **Блок 2:** мерч выводится как None, а последняя строка — «Тип суммы: <class 'int'>».
- **Блок 3:** после обмена — «Моё место: 13 | место друга: 12».

★ Добавь второго друга с местом 14 и сдвинь места по кругу одной строкой.

```
Моё место: 14 | места друзей: 12 13
```

★★ Школьникам скидка 20%: заведи константу DISCOUNT = 0.2 и посчитай сумму со скидкой. Объясни, почему изменился тип.

```
Со скидкой: 3600.0 руб.
Тип суммы: <class 'float'>
```

Челлендж: «Плейлист на вечер»

Собери программу-плейлист из трёх треков. Как она выглядит — решаешь сам. Требования:

- у каждого трека — название, исполнитель, длительность в секундах и «лайк»;
- общая длительность плейлиста считается из переменных;
- следующий трек после плейлиста пока не выбран;
- кнопка «перемешать»: треки сдвигаются по кругу, программа выводит новый порядок;
- в конце — типы трёх разных переменных через type().

Придумай, как вывести всё понятно и красиво — только через print.

Итоги темы 1.1

Теперь я умею…

- сохранять данные в переменных и называть их понятно: snake_case, константы заглавными;
- менять значение переменной и предсказывать, что в ней окажется;
- выбирать тип под данные — int, float, str, bool, None — и проверять его через type();
- замечать ловушки: "2" + "2", 0.1 + 0.2, опечатку в имени, и читать traceback снизу вверх;
- создавать несколько переменных одной строкой и обменивать значения без потерь.

Дальше — тема 1.2: программа начнёт спрашивать пользователя и красиво выводить ответ.